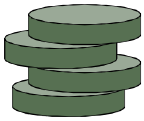
Stacks



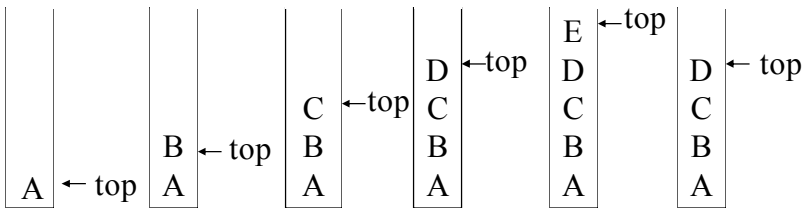
Queues

Stacks and Queues

- A stack is a last in, first out (LIFO) data structure
  - Items are removed from a stack in the reverse order from the way they were inserted
- A queue is a first in, first out (FIFO) data structure
  - Items are removed from a queue in the same order as they were inserted

Stack: Last In First Out

- A *stack* is a list with the restriction that insertions and deletions can be performed in only one position, namely, the *top* of the stack.
- The operations: **push** (insert) and **pop** (delete)



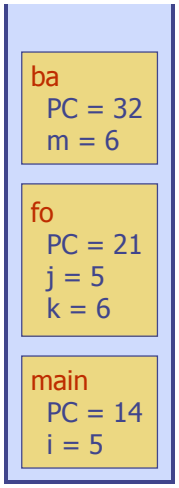
Applications of Stacks

- Direct applications
  - Page-visited history in a Web browser
  - Undo sequence in a text editor
  - Saving local variables when one function calls another, and this one calls another, and so on.
- Indirect applications
  - Auxiliary data structure for algorithms
  - Component of other data structures

Run-time Stack of C

- The C run-time system keeps track of the chain of active functions with a stack
- When a function is called, the run-time system pushes on the stack a frame containing
  - Local variables and return value
  - Program counter, keeping track of the statement being executed
- When a function returns, its frame is popped from the stack and control is passed to the function on top of the stack

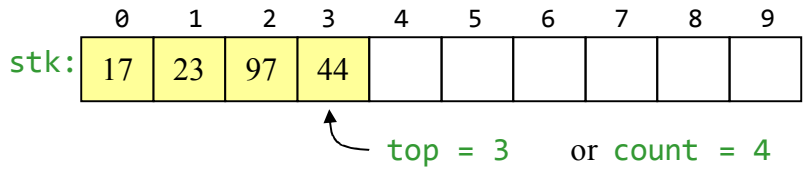
```
   main() {
     int i = 5;
14   fo(i);
   }

   fo(int j) {
     int k;
     k = j+1;
21   ba(k);
   }

32 ba(int m) {
     …
   }
```



Array Implementation of Stacks

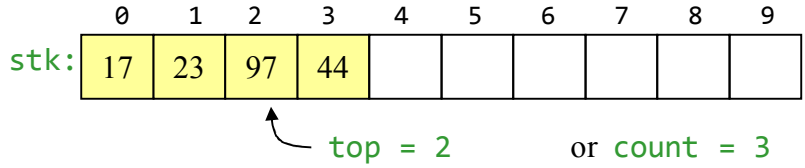
- To implement a stack, items are inserted and removed at the same end (called the top)
- To use an array to implement a stack, you need both the array itself and an integer
  - The integer tells you either:
    - Which location is currently the top of the stack, or
    - How many elements are in the stack

Push and Pop



- If the bottom of the stack is at location `0`, then an empty stack is represented by `top = -1` or `count = 0`
- To add (push) an element, either:
  - Increment `top` and store the element in `stk[top]`, or
  - Store the element in `stk[count]` and increment `count`
- To remove (pop) an element, either:
  - Get the element from `stk[top]` and decrement `top`, or
  - Decrement `count` and get the element in `stk[count]`

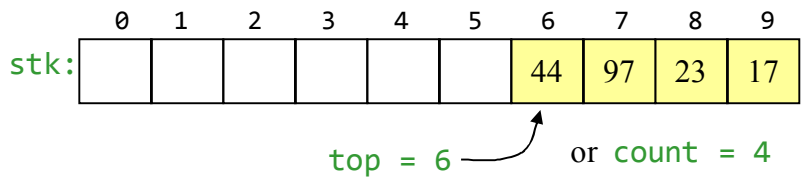
After Popping



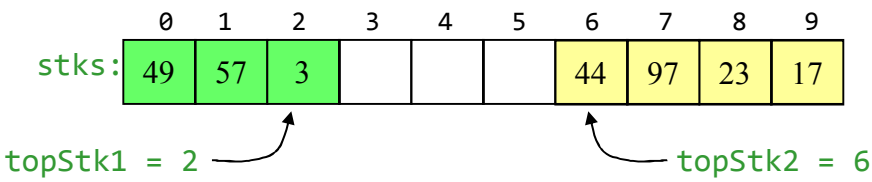
- When you pop an element, do you just leave the "deleted" element sitting in the array?
- The surprising answer is, *"it depends"*
  - If this is an array of primitives, *or* if you are programming in C or C++, *then* doing anything more is just a waste of time
  - If you are programming in Java, and the array contains objects, you should set the "deleted" array element to `null`
  - Why? To allow it to be garbage collected!

Sharing Space

- Of course, the bottom of the stack could be at the *other* end



- Sometimes this is done to allow two stacks to share the *same storage area*

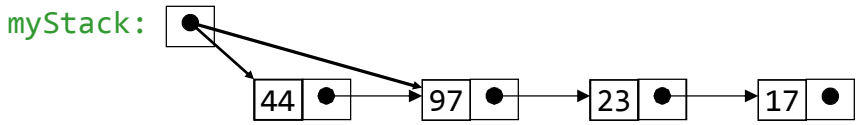


Error Checking

- There are two stack errors that can occur:
  - Underflow: trying to pop (or peek at) an empty stack
  - Overflow: trying to push onto an already full stack
- For underflow, you should throw an exception
  - If you don't catch it yourself, Java will throw an `ArrayIndexOutOfBounds` exception
  - You could create your own, more informative exception
- For overflow, you could do the same things
  - Or, you could check for the problem, and copy everything into a new, larger array

Linked-list Implementation of Stacks

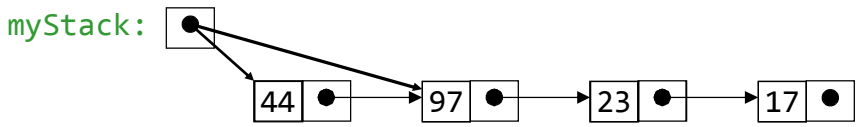
- Since all the actions happen at the top of a stack, a singly-linked list (SLL) is a fine way to implement it
- The header of the list points to the top of the stack



- Pushing is inserting an element at the front of the list

Linked-list Implementation of Stacks

- Since all the actions happen at the top of a stack, a singly-linked list (SLL) is a fine way to implement it
- The header of the list points to the top of the stack



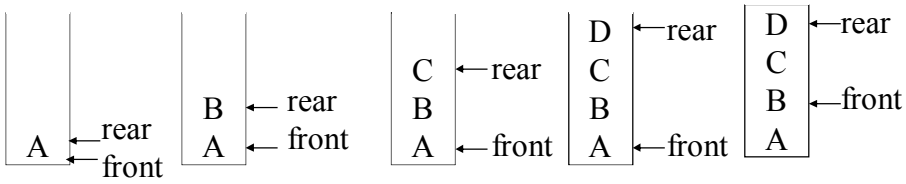
- Pushing is inserting an element at the front of the list
- Popping is removing an element from the front of the list

Linked-list Implementation Details

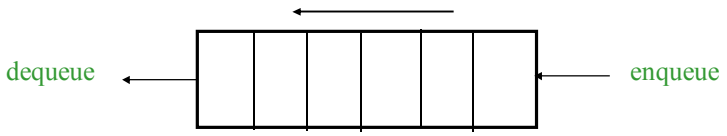
- With a linked-list representation, overflow will not happen (unless you exhaust memory, which is another kind of problem)
- Underflow can happen, and should be handled the same way as for an array implementation
- When a node is popped from a list, and the node references an object, the reference (the pointer in the node) does *not* need to be set to `null`
  - Unlike an array implementation, it really *is* removed--you can no longer get to it from the linked list
  - Hence, garbage collection can occur as appropriate

Queue: First In First Out

- A **Queue** is an ordered collection of items from which items may be removed at one end (called the ***front*** of the queue) and into which items may be inserted at the other end (the ***rear*** of the queue).



- The operations: **enqueue** (insert) and **dequeue** (delete)

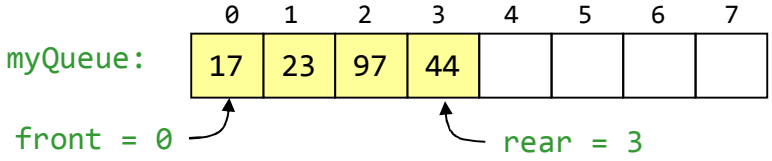


Applications of Queues

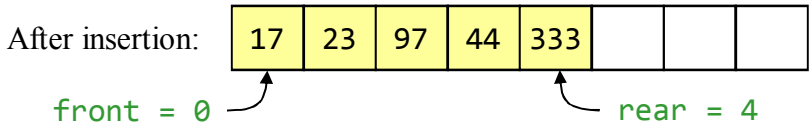
- Direct applications
  - Waiting lists, bureaucracy
  - Access to shared resources (e.g., printer)
  - Multiprogramming
- Indirect applications
  - Auxiliary data structure for algorithms
  - Component of other data structures

Array Implementation of Queues

- A queue is a first in, first out (FIFO) data structure
- This is accomplished by inserting at one end (the rear) and deleting from the other (the front)



- **To insert:** put new element in location 4, and set rear to 4



Array Implementation of Queues

- A queue is a first in, first out (FIFO) data structure
- This is accomplished by inserting at one end (the rear) and deleting from the other (the front)

After insertion:

17	23	97	44	333			

front = 0 rear = 4

- **To delete:** take element from location 0, and set front to 1

After deletion:

17	23	97	44	333			

front = 1 rear = 4

Array Implementation of Queues

front = 0 rear = 3

Initial queue:

17	23	97	44				

After insertion:

17	23	97	44	333			

After deletion:

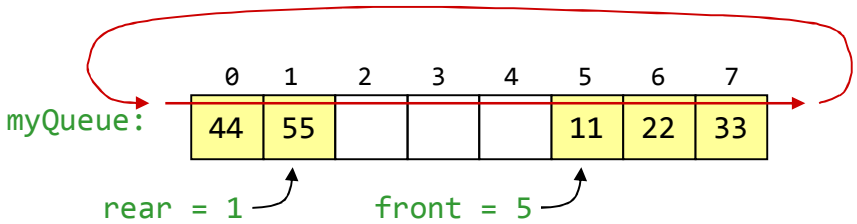
17	23	97	44	333			

front = 1 rear = 4

- Notice how the array contents "crawl" to the right as elements are inserted and deleted
- This will be a problem after a while!

Circular Queues using Arrays

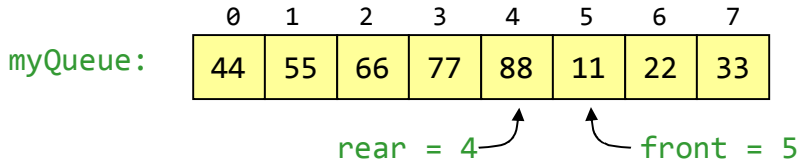
- We can treat the array holding the queue elements as circular (joined at the ends)



- Elements were added to this queue in the order 11, 22, 33, 44, 55, and will be removed in the same order
- Use: `front = (front + 1) % myQueue.length;`
  and: `rear = (rear + 1) % myQueue.length;`

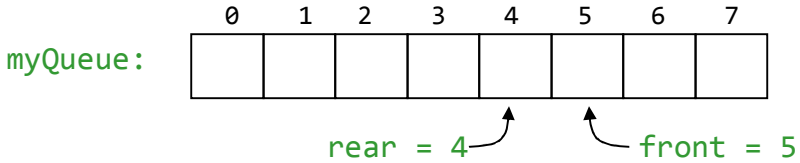
Full and Empty Queues

- If the queue were to become completely full, it would look like this:



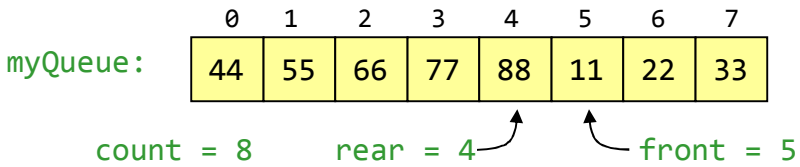
This is a problem!

- Again, if we were to remove all eight elements, making the queue completely empty, it would look like this:

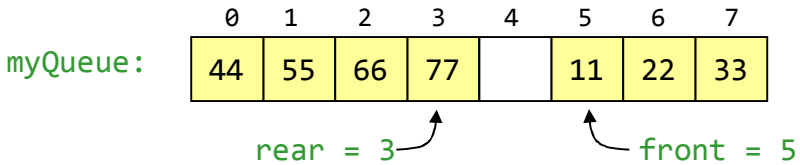


Full and Empty Queues: Solutions

- **Solution #1:** Keep an additional variable



- **Solution #2:** (Slightly more efficient) Keep a gap between elements: consider the queue full when it has `n-1` elements



Linked-list Implementation of Queues

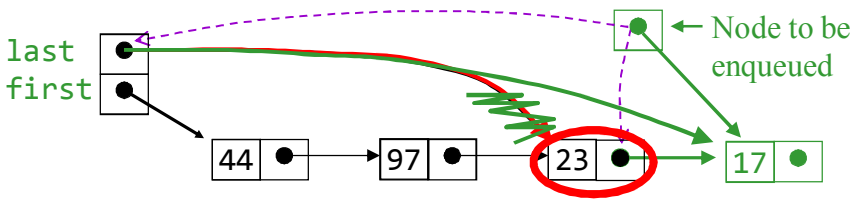
- In a queue, insertions occur at one end, deletions at the other end
- Operations at the front of a singly-linked list (SLL) are $O(1)$, but at the other end they are $O(n)$
  - Because you have to find the last element each time
- BUT: there is a simple way to use a singly-linked list to implement both insertions and deletions in $O(1)$ time
  - You always need a pointer to the first thing in the list
  - You can keep an additional pointer to the *last* thing in the list

Linked-list Implementation of Queues

SLL Implementation of Queues

- In an SLL you can easily find the successor of a node, but not its predecessor
  - Remember, pointers (references) are one-way
- If you know where the *last* node in a list is, it's hard to remove that node, but it's easy to add a node after it
- Hence,
  - Use the *first* element in an SLL as the *front* of the queue
  - Use the *last* element in an SLL as the *rear* of the queue
  - Keep pointers to *both* the front and the rear of the SLL

Enqueueing a Node



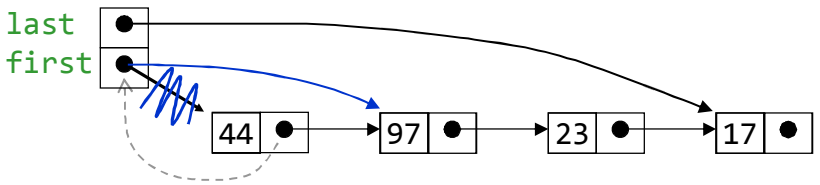
To enqueue (add) a node:

Find the current last node

Change it to point to the new last node

Change the `last` pointer in the list header

Dequeueing a Node



- To dequeue (remove) a node:
  - Copy the pointer from the first node into the header

Queue Implementation Details

- With an array implementation:
  - you can have both overflow and underflow
- With a linked-list implementation:
  - you can have underflow
  - overflow is a global out-of-memory condition